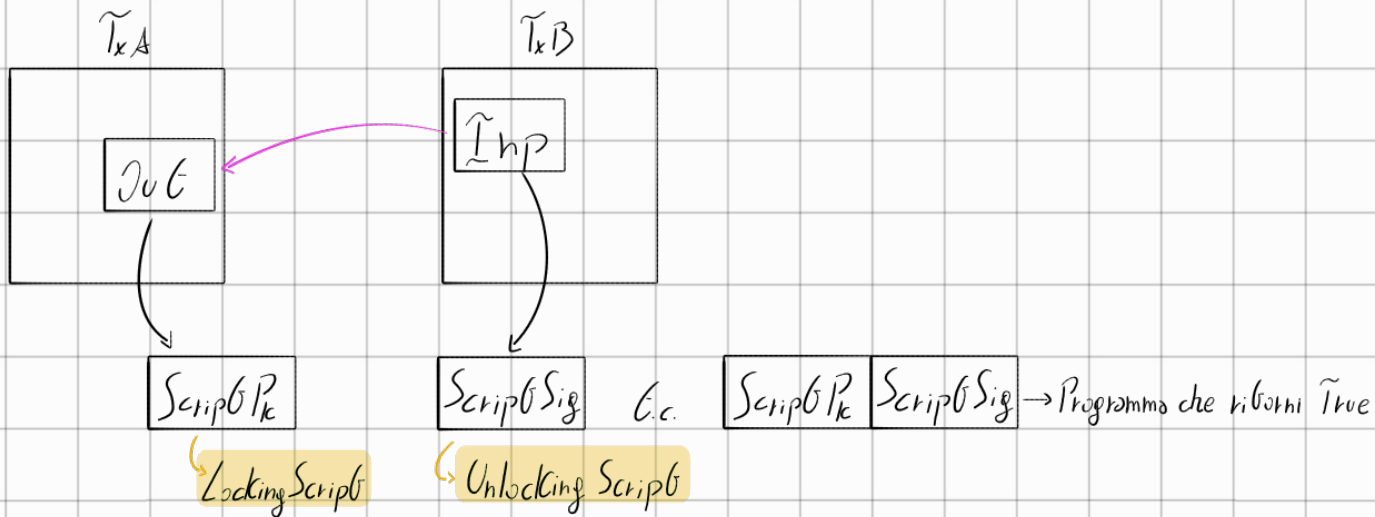


Script

Abbiamo visto che le transazioni vengono validate mediante l'utilizzo di LockingScript e UnlockingScript

Questo meccanismo è stato progettato in modo tale che una transazione che voglia spendere un output debba contenere, all'interno di uno dei suoi input, un UnlockingScript c.c. la concatenazione tra UnlockingScript e LockingScript restituisca True



Linguaggio Script

Il software del sistema Bitcoin è in modo che all'interno di ogni nodo sia presente l'interprete del linguaggio Script

Questo linguaggio è

- 1) stateless → senza possibilità di utilizzare delle variabili
- 2) stack based → dati ed istruzioni vengono letti ed eseguiti come se fossero all'interno di una pila

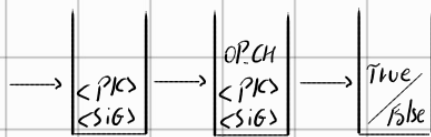
Un programma è un insieme di istruzioni e dati che vengono eseguiti su un modello stack based

Per quanto riguarda le istruzioni, queste possono estrarre ed immettere dati all'interno della pila

Esempio

OP_ADD = prende i due elementi sottostanti della pila e ne calcola la somma





oss

Si può notare come la <pk> venga comunque resa pubblica nello Script Sig,
Per questo di solito ogni coppia di chiavi viene usata una sola volta

Pay To Multisig (P2m)

Può essere necessario a volte che l'output di una transazione possa essere speso solo se due o più persone si mettono d'accordo

ϵ $z <pk_A> <pk_B> <pk_C> 3$ $OP_CHECKMULTISIG$ n

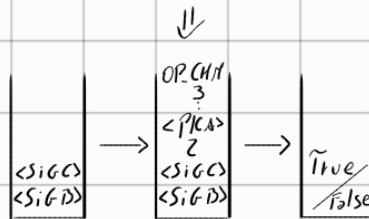
Prende un intero n , n chiavi pubbliche, un intero ϵ , ϵ firme e verifica che le firme siano valide rispetto alle n chiavi in input

Per cui il rispettivo Unlocking script sarà della forma

$<Sig_B> <Sig_C>$

In questo modo si ha:

$<Sig_B> <Sig_C> z <pk_A> <pk_B> <pk_C> 3$ $OP_CHECKMULTISIG$



oss

Ovviamente si può facilmente progettare uno Script di tipo Multisig che permetta di usare l'hash delle PK piuttosto che lasciarle in chiaro

Pay To Script Hash (P2Sh)

All'interno dei precedenti schemi è il mittente che decide le condizioni con cui il destinatario possa spendere l'output

Per assicurarne questo compito al destinatario piuttosto che al mittente sono nati gli Script P2Sh dove è il destinatario che crea i redeem script e ne invia l'hash al mittente che lo inserisce nello Script PK

$OP_HASH160 <redeem_script_hash> OP_EQUAL$

Per cui il destinatario dovrà inserire all'interno dello ScriptSig il redeem Script seguito dalle condizioni necessarie per renderlo True

↳ Dopo aver verificato che l'hash del redeem corrisponde,

questo viene eseguito

Salvare dati nella blockchain

Essendo la blockchain di bitcoin un registro permanente ed immutabile, si è cominciato ad usare i byte riservati agli script per salvare dei dati.

↳ Unspent Transaction Output

Per evitare che questi vengano salvati all'interno dell'UTxO è stata introdotta l'istruzione `OP_RETURN`, che fa terminare qualsiasi script che la contenga.